📋 Rapport Technique — Processus Scan/Print Étiquettes QR Code CMDB Inventory

Version: 2.0
Date: 26 Mars 2026
Auteur: Équipe Technique CMDB
Statut: Production-Ready

📑 Table des Matières

1. Analyse du Code Actuel
2. Architecture Améliorée
3. Composants Backend
4. Endpoints DRF
5. URLs Configuration
6. Workflow Complet
7. Procédure d'Utilisation

---

1. Analyse du Code Actuel

1.1 Problèmes Identifiés

Problème	Impact	Priorité
Format PDF A4	Incompatible imprimantes thermiques	🔴 Critique
Import Image manquant	Erreur d'exécution	🔴 Critique
Pas de batch printing	1 asset = 1 PDF (lent)	🟠 Haute
Pas de template système	Layout fixe non flexible	🟠 Haute
Pas de print log	Aucun audit/tracking	🟠 Haute
Synchrone	Bloquant pour gros batches	🟡 Moyenne
Pas de gestion imprimante	Configuration manuelle	🟡 Moyenne
QR position hardcoded	Non adaptable formats	🟡 Moyenne

1.2 Code Corrigé (Base)

```python
# backend/scanner/views.py (VERSION CORRIGÉE)
from django.http import HttpResponse
from django.shortcuts import get_object_or_404
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import mm
from reportlab.lib.utils import ImageReader
from PIL import Image
from io import BytesIO
from scanner.models import QRCode
from inventory.models import Asset

def print_label_pdf_view(request, asset_id):
    """Génère un PDF d'étiquette pour impression thermique."""
    asset = get_object_or_404(Asset, id=asset_id)

    # Format étiquette thermique (80mm x 50mm)
    width, height = 80 * mm, 50 * mm
    buffer = BytesIO()
    p = canvas.Canvas(buffer, pagesize=(width, height))

    # QR Code
    try:
        qr_code = QRCode.objects.get(asset=asset)
        if qr_code.image:
            qr_img = ImageReader(qr_code.image.path)
            p.drawImage(qr_img, 5*mm, 10*mm, width=30*mm, height=30*mm)
    except QRCode.DoesNotExist:
        pass

    # Texte
    p.setFont("Helvetica-Bold", 10)
    p.drawString(40*mm, 40*mm, asset.name[:30])
    p.setFont("Helvetica", 8)
    p.drawString(40*mm, 35*mm, f"S/N: {asset.serial_number}")
    p.drawString(40*mm, 30*mm, f"ID: {asset.internal_code}")

    p.showPage()
    p.save()
    buffer.seek(0)

    response = HttpResponse(buffer, content_type='application/pdf')
    response['Content-Disposition'] = f'attachment; filename="label_{asset.id}.pdf"'
    return response
```

2. Architecture Améliorée

2.1 Nouveaux Modèles

```python
# backend/scanner/models.py
import uuid
from django.db import models
from django.conf import settings
from datetime import timedelta


class PrintTemplate(models.Model):
    """
    Template d'étiquette configurable.
    Permet différents formats selon le type d'asset.
    """
    FORMAT_CHOICES = [
        ('30x20', '30mm x 20mm (Petit matériel)'),
        ('50x30', '50mm x 30mm (Standard)'),
        ('70x40', '70mm x 40mm (Grand matériel)'),
        ('80x50', '80mm x 50mm (Rack/Serveur)'),
        ('100x50', '100mm x 50mm (Armoire)'),
    ]

    name = models.CharField(max_length=100)
    format = models.CharField(max_length=20, choices=FORMAT_CHOICES, default='50x30')
    width_mm = models.PositiveIntegerField(default=50)
    height_mm = models.PositiveIntegerField(default=30)

    # Layout configuration
    qr_size_mm = models.PositiveIntegerField(default=25)
    qr_position_x_mm = models.PositiveIntegerField(default=5)
    qr_position_y_mm = models.PositiveIntegerField(default=5)

    font_size_title = models.PositiveIntegerField(default=10)
    font_size_text = models.PositiveIntegerField(default=8)

    # Fields to display
    show_serial = models.BooleanField(default=True)
    show_internal_code = models.BooleanField(default=True)
    show_category = models.BooleanField(default=True)
    show_location = models.BooleanField(default=True)
    show_purchase_date = models.BooleanField(default=False)
    show_warranty = models.BooleanField(default=False)

    is_default = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-is_default', '-created_at']

    def __str__(self):
```

```python
        return f"{self.name} ({self.format})"


class Printer(models.Model):
    """
    Configuration des imprimantes thermiques.
    Supporte USB, Bluetooth, Ethernet.
    """
    CONNECTION_CHOICES = [
        ('usb', 'USB'),
        ('bluetooth', 'Bluetooth'),
        ('ethernet', 'Ethernet/WiFi'),
        ('network', 'Network Printer'),
    ]

    name = models.CharField(max_length=100)
    connection_type = models.CharField(max_length=20, choices=CONNECTION_CHOICES)

    # Network config
    ip_address = models.GenericIPAddressField(null=True, blank=True)
    port = models.PositiveIntegerField(default=9100)

    # USB/Bluetooth
    device_path = models.CharField(max_length=255, blank=True)
    mac_address = models.CharField(max_length=17, blank=True)

    # Printer settings
    dpi = models.PositiveIntegerField(default=203)
    speed = models.PositiveIntegerField(default=3)  # 1-5
    density = models.PositiveIntegerField(default=10)  # 1-15

    # Paper
    default_template = models.ForeignKey(PrintTemplate, on_delete=models.SET_NULL,
null=True)
    paper_width_mm = models.PositiveIntegerField(default=50)
    paper_height_mm = models.PositiveIntegerField(default=30)

    is_active = models.BooleanField(default=True)
    is_default = models.BooleanField(default=False)
    last_used_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-is_default', '-created_at']

    def __str__(self):
        return f"{self.name} ({self.get_connection_type_display()})"


class PrintJob(models.Model):
    """
    Job d'impression (batch ou single).
    Track le statut et l'historique.
```

```
    """
    STATUS_CHOICES = [
        ('pending', 'En attente'),
        ('processing', 'En cours'),
        ('completed', 'Terminé'),
        ('failed', 'Échoué'),
        ('cancelled', 'Annulé'),
    ]

    uuid = models.UUIDField(default=uuid.uuid4, unique=True)
    created_by = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.SET_NULL,
null=True)

    asset_ids = models.JSONField(help_text="Liste des asset IDs à imprimer")
    template = models.ForeignKey(PrintTemplate, on_delete=models.SET_NULL, null=True)
    printer = models.ForeignKey(Printer, on_delete=models.SET_NULL, null=True)

    copies = models.PositiveIntegerField(default=1)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending')

    pdf_file = models.FileField(upload_to='print_jobs/%Y/%m/', blank=True, null=True)

    error_message = models.TextField(blank=True)
    started_at = models.DateTimeField(null=True, blank=True)
    completed_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-created_at']

    def __str__(self):
        return f"PrintJob #{self.id} - {self.status}"

    @property
    def asset_count(self):
        return len(self.asset_ids) if self.asset_ids else 0


class PrintLog(models.Model):
    """
    Log d'audit pour chaque impression.
    Compliance et traçabilité.
    """
    job = models.ForeignKey(PrintJob, on_delete=models.CASCADE, related_name='logs')
    asset = models.ForeignKey('inventory.Asset', on_delete=models.CASCADE)
    printed_by = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.SET_NULL,
null=True)

    printer_name = models.CharField(max_length=100)
    template_name = models.CharField(max_length=100)

    printed_at = models.DateTimeField(auto_now_add=True)
    ip_address = models.GenericIPAddressField(null=True, blank=True)
```

```
    user_agent = models.TextField(blank=True)

    class Meta:
        ordering = ['-printed_at']
        indexes = [
            models.Index(fields=['asset', '-printed_at']),
            models.Index(fields=['printed_by', '-printed_at']),
        ]
```

---

3. Composants Backend

3.1 Service PDF (backend/scanner/services/pdf_generator.py)

```
# backend/scanner/services/pdf_generator.py
"""
Service de génération PDF pour étiquettes QR Code.
Supporte multiple formats et templates.
"""

import io
import logging
from datetime import datetime
from decimal import Decimal
from django.conf import settings
from PIL import Image
from reportlab.lib import colors
from reportlab.lib.pagesizes import mm
from reportlab.lib.units import mm as reportlab_mm
from reportlab.pdfgen import canvas
from reportlab.platypus import Table, TableStyle, Paragraph
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import TA_CENTER, TA_LEFT
from reportlab.graphics.barcode import code128, qr

logger = logging.getLogger(__name__)


class PDFLabelGenerator:
    """Générateur de PDF pour étiquettes d'assets."""

    def __init__(self, template, printer=None):
        """
        Initialize le générateur avec template et imprimante.

        Args:
            template: PrintTemplate instance
            printer: Printer instance (optionnel)
        """
        self.template = template
        self.printer = printer
        self.width = template.width_mm * mm
```

```
        self.height = template.height_mm * mm

    def generate_single(self, asset, copies=1):
        """
        Génère PDF pour un seul asset.

        Args:
            asset: Asset instance
            copies: Nombre de copies

        Returns:
            BytesIO: Buffer PDF
        """
        buffer = io.BytesIO()
        p = canvas.Canvas(buffer, pagesize=(self.width, self.height))

        for i in range(copies):
            self._draw_page(p, asset)
            if i < copies - 1:
                p.showPage()

        p.save()
        buffer.seek(0)
        return buffer

    def generate_batch(self, assets, copies=1):
        """
        Génère PDF pour multiple assets.

        Args:
            assets: QuerySet ou list d'assets
            copies: Nombre de copies par asset

        Returns:
            BytesIO: Buffer PDF
        """
        buffer = io.BytesIO()
        p = canvas.Canvas(buffer, pagesize=(self.width, self.height))

        first_page = True
        for asset in assets:
            for i in range(copies):
                if not first_page:
                    p.showPage()
                first_page = False
                self._draw_page(p, asset)

        p.save()
        buffer.seek(0)
        return buffer

    def _draw_page(self, canvas_obj, asset):
        """Dessine une page/étiquette."""
```

```python
        # QR Code
        try:
            qr_code = asset.qrcode
            if qr_code and qr_code.image:
                self._draw_qr_code(canvas_obj, qr_code.image.path)
        except Exception as e:
            logger.warning(f"QR Code non disponible pour asset {asset.id}: {e}")

        # Texte
        self._draw_text(canvas_obj, asset)

        # Barcode (optionnel)
        if self.template.show_serial and asset.serial_number:
            self._draw_barcode(canvas_obj, asset.serial_number)

    def _draw_qr_code(self, canvas_obj, image_path):
        """Dessine le QR Code."""
        try:
            from reportlab.lib.utils import ImageReader
            qr_img = ImageReader(image_path)

            x = self.template.qr_position_x_mm * mm
            y = self.template.qr_position_y_mm * mm
            size = self.template.qr_size_mm * mm

            canvas_obj.drawImage(
                qr_img, x, y,
                width=size, height=size,
                preserveAspectRatio=True
            )
        except Exception as e:
            logger.error(f"Erreur dessin QR Code: {e}")

    def _draw_text(self, canvas_obj, asset):
        """Dessine le texte de l'étiquette."""
        # Styles
        title_style = ParagraphStyle(
            'CustomTitle',
            parent=getSampleStyleSheet()['Bold'],
            fontSize=self.template.font_size_title,
            alignment=TA_LEFT,
            leading=self.template.font_size_title + 2
        )

        text_style = ParagraphStyle(
            'CustomText',
            parent=getSampleStyleSheet()['Normal'],
            fontSize=self.template.font_size_text,
            alignment=TA_LEFT,
            leading=self.template.font_size_text + 2
        )

        # Position texte (à droite du QR)
```

```
        text_x = (self.template.qr_position_x_mm + self.template.qr_size_mm + 5) * mm
        text_y = self.height - 10 * mm

        # Titre (Nom asset)
        canvas_obj.drawString(
            text_x, text_y,
            asset.name[:40]  # Limiter longueur
        )
        text_y -= (self.template.font_size_title + 4) * mm

        # Serial Number
        if self.template.show_serial and asset.serial_number:
            canvas_obj.drawString(
                text_x, text_y,
                f"S/N: {asset.serial_number}"
            )
            text_y -= (self.template.font_size_text + 2) * mm

        # Internal Code
        if self.template.show_internal_code and asset.internal_code:
            canvas_obj.drawString(
                text_x, text_y,
                f"ID: {asset.internal_code}"
            )
            text_y -= (self.template.font_size_text + 2) * mm

        # Category
        if self.template.show_category and asset.category:
            canvas_obj.drawString(
                text_x, text_y,
                f"Cat: {asset.category.name}"
            )
            text_y -= (self.template.font_size_text + 2) * mm

        # Location
        if self.template.show_location and asset.current_location:
            canvas_obj.drawString(
                text_x, text_y,
                f"Loc: {asset.current_location.name}"
            )
            text_y -= (self.template.font_size_text + 2) * mm

        # Purchase Date
        if self.template.show_purchase_date and asset.purchase_date:
            canvas_obj.drawString(
                text_x, text_y,
                f"Achat: {asset.purchase_date.strftime('%d/%m/%Y')}"
            )
            text_y -= (self.template.font_size_text + 2) * mm

        # Warranty
        if self.template.show_warranty and asset.warranty_end:
            canvas_obj.drawString(
```

```
                text_x, text_y,
                f"Garantie: {asset.warranty_end.strftime('%d/%m/%Y')}"
            )

    def _draw_barcode(self, canvas_obj, serial_number):
        """Dessine le code-barres (optionnel, en bas)."""
        try:
            barcode = code128.Code128(
                serial_number,
                barHeight=15 * mm,
                barWidth=0.4 * mm
            )
            barcode.drawOn(
                canvas_obj,
                self.template.qr_position_x_mm * mm,
                5 * mm
            )
        except Exception as e:
            logger.warning(f"Erreur dessin barcode: {e}")


def generate_label_pdf(assets, template, printer=None, copies=1):
    """
    Fonction utilitaire pour générer PDF d'étiquettes.

    Args:
        assets: List ou QuerySet d'assets
        template: PrintTemplate
        printer: Printer (optionnel)
        copies: Nombre de copies

    Returns:
        BytesIO: Buffer PDF
    """
    generator = PDFLabelGenerator(template, printer)

    if hasattr(assets, '__iter__') and not isinstance(assets, str):
        return generator.generate_batch(assets, copies)
    else:
        return generator.generate_single(assets, copies)
```

3.2 Celery Tasks (backend/scanner/tasks.py)

```
# backend/scanner/tasks.py
"""
Tasks Celery pour génération PDF asynchrone.
Permet de traiter les gros batches sans bloquer.
"""

import logging
from celery import shared_task
from django.core.files.base import ContentFile
```

```python
from django.utils import timezone
from .models import PrintJob, PrintTemplate, Printer, PrintLog
from .services.pdf_generator import generate_label_pdf
from inventory.models import Asset

logger = logging.getLogger(__name__)


@shared_task(bind=True, max_retries=3)
def generate_print_job_pdf(self, job_id):
    """
    Génère le PDF pour un PrintJob (async).

    Usage:
        generate_print_job_pdf.delay(job_id)
    """
    try:
        job = PrintJob.objects.select_related('template', 'printer').get(id=job_id)
        job.status = 'processing'
        job.started_at = timezone.now()
        job.save(update_fields=['status', 'started_at'])

        # Récupérer les assets
        assets = Asset.objects.filter(id__in=job.asset_ids)

        if not assets.exists():
            raise ValueError(f"Aucun asset trouvé pour IDs: {job.asset_ids}")

        # Générer PDF
        pdf_buffer = generate_label_pdf(
            assets=assets,
            template=job.template,
            printer=job.printer,
            copies=job.copies
        )

        # Sauvegarder PDF
        from django.core.files.storage import default_storage
        filename = f"print_jobs/{timezone.now().strftime('%Y/%m')}/job_{job.uuid}.pdf"

        job.pdf_file.save(
            filename,
            ContentFile(pdf_buffer.getvalue()),
            save=True
        )

        # Créer logs d'audit
        for asset in assets:
            PrintLog.objects.create(
                job=job,
                asset=asset,
                printed_by=job.created_by,
                printer_name=job.printer.name if job.printer else 'N/A',
```

```
                template_name=job.template.name if job.template else 'N/A',
                ip_address=None,  # Peut être ajouté depuis request
                user_agent=None
            )

        # Mettre à jour statut
        job.status = 'completed'
        job.completed_at = timezone.now()
        job.save(update_fields=['status', 'completed_at'])

        logger.info(f"PrintJob {job_id} terminé avec succès")
        return {'status': 'success', 'job_id': job_id, 'pdf_url': job.pdf_file.url}

    except Exception as e:
        logger.error(f"Erreur PrintJob {job_id}: {str(e)}", exc_info=True)

        job.status = 'failed'
        job.error_message = str(e)
        job.save(update_fields=['status', 'error_message'])

        # Retry automatique
        raise self.retry(exc=e, countdown=60)


@shared_task
def cleanup_old_print_jobs(days=30):
    """
    Nettoie les anciens PrintJobs (maintenance).

    Usage:
        cleanup_old_print_jobs.delay(days=30)
    """
    from datetime import timedelta
    cutoff = timezone.now() - timedelta(days=days)

    old_jobs = PrintJob.objects.filter(created_at__lt=cutoff)
    count = old_jobs.count()

    # Supprimer fichiers PDF
    for job in old_jobs:
        if job.pdf_file:
            job.pdf_file.delete(save=False)

    old_jobs.delete()

    logger.info(f"Nettoyage: {count} PrintJobs supprimés")
    return {'deleted': count}


@shared_task
def generate_missing_qr_codes():
    """
    Génère les QR Codes manquants pour les assets.
```

```
    Usage:
        generate_missing_qr_codes.delay()
    """
    from .signals import generate_missing_qr_codes as sync_func
    result = sync_func()
    return result
```

3.3 Management Commands

```
# backend/scanner/management/commands/generate_labels.py
"""
Management command pour génération batch d'étiquettes.
"""

from django.core.management.base import BaseCommand, CommandError
from scanner.models import PrintJob, PrintTemplate
from inventory.models import Asset
from scanner.tasks import generate_print_job_pdf


class Command(BaseCommand):
    help = 'Génère des étiquettes PDF pour des assets'

    def add_arguments(self, parser):
        parser.add_argument(
            '--assets',
            type=int,
            nargs='+',
            help='Asset IDs à imprimer'
        )
        parser.add_argument(
            '--category',
            type=str,
            help='Catégorie d\'assets (imprime tous les assets de cette catégorie)'
        )
        parser.add_argument(
            '--template',
            type=int,
            default=1,
            help='Template ID (défaut: 1)'
        )
        parser.add_argument(
            '--copies',
            type=int,
            default=1,
            help='Nombre de copies (défaut: 1)'
        )
        parser.add_argument(
            '--async',
            action='store_true',
            help='Générer en async (Celery)'
```

```
        )
        parser.add_argument(
            '--output',
            type=str,
            help='Chemin de sortie du PDF'
        )

    def handle(self, *args, **options):
        asset_ids = options['assets']
        category = options['category']
        template_id = options['template']
        copies = options['copies']
        use_async = options['async']

        # Récupérer assets
        if asset_ids:
            assets = Asset.objects.filter(id__in=asset_ids)
        elif category:
            from inventory.models import Category
            cat = Category.objects.filter(name__icontains=category).first()
            if not cat:
                raise CommandError(f'Catégorie non trouvée: {category}')
            assets = Asset.objects.filter(category=cat)
        else:
            raise CommandError('Spécifier --assets ou --category')

        if not assets.exists():
            raise CommandError('Aucun asset trouvé')

        # Récupérer template
        try:
            template = PrintTemplate.objects.get(id=template_id)
        except PrintTemplate.DoesNotExist:
            raise CommandError(f'Template non trouvé: {template_id}')

        self.stdout.write(f'📊 {assets.count()} assets à imprimer')
        self.stdout.write(f'📄 Template: {template.name}')
        self.stdout.write(f'📋 Copies: {copies}')

        # Créer PrintJob
        job = PrintJob.objects.create(
            asset_ids=list(assets.values_list('id', flat=True)),
            template=template,
            copies=copies,
            status='pending'
        )

        if use_async:
            # Async avec Celery
            self.stdout.write('🚀 Lancement task Celery...')
            generate_print_job_pdf.delay(job.id)
            self.stdout.write(self.style.SUCCESS(f'✅ Job {job.id} en cours (async)'))
        else:
```

```
            # Sync
            from scanner.services.pdf_generator import generate_label_pdf
            pdf_buffer = generate_label_pdf(assets, template, copies=copies)

            output_path = options['output'] or f'/tmp/labels_{job.id}.pdf'
            with open(output_path, 'wb') as f:
                f.write(pdf_buffer.getvalue())

            job.status = 'completed'
            job.save()

            self.stdout.write(self.style.SUCCESS(f'✅ PDF généré: {output_path}'))
```

---

4. Endpoints DRF

4.1 ViewSets (backend/scanner/api/views.py)

```
# backend/scanner/api/views.py
"""
API Endpoints pour impression d'étiquettes.
"""

from rest_framework import viewsets, status, decorators
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from rest_framework.parsers import MultiPartParser, FormParser
from django.http import HttpResponse
from django.shortcuts import get_object_or_404
from django.utils import timezone
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.filters import SearchFilter, OrderingFilter

from scanner.models import PrintTemplate, Printer, PrintJob, PrintLog
from scanner.api.serializers import (
    PrintTemplateSerializer,
    PrinterSerializer,
    PrintJobSerializer,
    PrintLogSerializer
)
from scanner.tasks import generate_print_job_pdf
from scanner.services.pdf_generator import generate_label_pdf
from inventory.models import Asset


class PrintTemplateViewSet(viewsets.ModelViewSet):
    """
    CRUD Templates d'étiquettes.

    list: GET /api/v1/scanner/templates/
    create: POST /api/v1/scanner/templates/
    retrieve: GET /api/v1/scanner/templates/<id>/
```

```
    update: PUT /api/v1/scanner/templates/<id>/
    destroy: DELETE /api/v1/scanner/templates/<id>/
    """
    queryset = PrintTemplate.objects.all()
    serializer_class = PrintTemplateSerializer
    permission_classes = [IsAuthenticated]
    filter_backends = [DjangoFilterBackend, OrderingFilter]
    filterset_fields = ['format', 'is_default']
    ordering_fields = ['created_at', 'name']


class PrinterViewSet(viewsets.ModelViewSet):
    """
    CRUD Imprimantes.

    list: GET /api/v1/scanner/printers/
    create: POST /api/v1/scanner/printers/
    retrieve: GET /api/v1/scanner/printers/<id>/
    update: PUT /api/v1/scanner/printers/<id>/
    destroy: DELETE /api/v1/scanner/printers/<id>/
    """
    queryset = Printer.objects.all()
    serializer_class = PrinterSerializer
    permission_classes = [IsAuthenticated]
    filter_backends = [DjangoFilterBackend, OrderingFilter]
    filterset_fields = ['connection_type', 'is_active', 'is_default']
    ordering_fields = ['created_at', 'name']


class PrintJobViewSet(viewsets.ModelViewSet):
    """
    Gestion des jobs d'impression.

    list: GET /api/v1/scanner/print-jobs/
    create: POST /api/v1/scanner/print-jobs/
    retrieve: GET /api/v1/scanner/print-jobs/<id>/
    download: GET /api/v1/scanner/print-jobs/<id>/download/
    cancel: POST /api/v1/scanner/print-jobs/<id>/cancel/
    """
    queryset = PrintJob.objects.all()
    serializer_class = PrintJobSerializer
    permission_classes = [IsAuthenticated]
    filter_backends = [DjangoFilterBackend, SearchFilter, OrderingFilter]
    filterset_fields = ['status', 'created_by']
    search_fields = ['uuid', 'asset_ids']
    ordering_fields = ['created_at', 'completed_at']

    @decorators.action(detail=True, methods=['get'])
    def download(self, request, pk=None):
        """Télécharger le PDF du PrintJob."""
        job = self.get_object()

        if job.status != 'completed':
```

```
            return Response(
                {'error': 'Job non terminé'},
                status=status.HTTP_400_BAD_REQUEST
            )

        if not job.pdf_file:
            return Response(
                {'error': 'PDF non disponible'},
                status=status.HTTP_404_NOT_FOUND
            )

        response = HttpResponse(job.pdf_file.read(), content_type='application/pdf')
        response['Content-Disposition'] = f'attachment; filename="labels_{job.uuid}.pdf"'
        return response

    @decorators.action(detail=True, methods=['post'])
    def cancel(self, request, pk=None):
        """Annuler un PrintJob."""
        job = self.get_object()

        if job.status in ['completed', 'cancelled']:
            return Response(
                {'error': 'Job déjà terminé ou annulé'},
                status=status.HTTP_400_BAD_REQUEST
            )

        job.status = 'cancelled'
        job.save()

        return Response({'status': 'cancelled'})

    @decorators.action(detail=False, methods=['post'])
    def generate(self, request):
        """
        Générer un nouveau PrintJob.

        POST /api/v1/scanner/print-jobs/generate/
        Body:
        {
            "asset_ids": [152, 153, 154],
            "template_id": 1,
            "printer_id": 1,
            "copies": 1,
            "async": true
        }
        """
        asset_ids = request.data.get('asset_ids', [])
        template_id = request.data.get('template_id')
        printer_id = request.data.get('printer_id')
        copies = request.data.get('copies', 1)
        use_async = request.data.get('async', True)

        if not asset_ids:
```

```
            return Response(
                {'error': 'asset_ids requis'},
                status=status.HTTP_400_BAD_REQUEST
            )

        # Valider assets
        assets = Asset.objects.filter(id__in=asset_ids)
        if len(assets) != len(asset_ids):
            return Response(
                {'error': 'Certains assets n\'existent pas'},
                status=status.HTTP_400_BAD_REQUEST
            )

        # Template
        template = None
        if template_id:
            template = get_object_or_404(PrintTemplate, id=template_id)
        else:
            template = PrintTemplate.objects.filter(is_default=True).first()

        # Printer
        printer = None
        if printer_id:
            printer = get_object_or_404(Printer, id=printer_id)

        # Créer PrintJob
        job = PrintJob.objects.create(
            created_by=request.user,
            asset_ids=asset_ids,
            template=template,
            printer=printer,
            copies=copies,
            status='pending'
        )

        if use_async:
            # Async avec Celery
            generate_print_job_pdf.delay(job.id)
            return Response({
                'status': 'pending',
                'job_id': job.id,
                'uuid': str(job.uuid),
                'message': 'Job en cours de traitement'
            })
        else:
            # Sync
            pdf_buffer = generate_label_pdf(assets, template, printer, copies)

            from django.core.files.base import ContentFile
            filename = f"print_jobs/{timezone.now().strftime('%Y/%m')}/job_{job.uuid}.pdf"
            job.pdf_file.save(filename, ContentFile(pdf_buffer.getvalue()), save=True)

            job.status = 'completed'
```

```
            job.completed_at = timezone.now()
            job.save()

            return Response({
                'status': 'completed',
                'job_id': job.id,
                'uuid': str(job.uuid),
                'pdf_url': job.pdf_file.url
            })


class PrintLogViewSet(viewsets.ReadOnlyModelViewSet):
    """
    Logs d'impression (lecture seule).

    list: GET /api/v1/scanner/print-logs/
    retrieve: GET /api/v1/scanner/print-logs/<id>/
    """
    queryset = PrintLog.objects.all()
    serializer_class = PrintLogSerializer
    permission_classes = [IsAuthenticated]
    filter_backends = [DjangoFilterBackend, SearchFilter, OrderingFilter]
    filterset_fields = ['asset', 'printed_by', 'job']
    ordering_fields = ['printed_at']
    ordering = ['-printed_at']


# Endpoint rapide pour impression directe (single asset)
@decorators.api_view(['GET'])
@decorators.permission_classes([IsAuthenticated])
def print_label_direct(request, asset_id):
    """
    Impression directe d'un asset (sans PrintJob).

    GET /api/v1/scanner/print-label/<asset_id>/
    """
    asset = get_object_or_404(Asset, id=asset_id)
    template = PrintTemplate.objects.filter(is_default=True).first()

    if not template:
        template = PrintTemplate.objects.first()

    pdf_buffer = generate_label_pdf([asset], template, copies=1)

    response = HttpResponse(pdf_buffer.read(), content_type='application/pdf')
    response['Content-Disposition'] = f'attachment; filename="label_{asset.id}.pdf"'
    return response
```

4.2 Serializers (backend/scanner/api/serializers.py)

```
# backend/scanner/api/serializers.py
from rest_framework import serializers
from scanner.models import PrintTemplate, Printer, PrintJob, PrintLog


class PrintTemplateSerializer(serializers.ModelSerializer):
    class Meta:
        model = PrintTemplate
        fields = '__all__'


class PrinterSerializer(serializers.ModelSerializer):
    class Meta:
        model = Printer
        fields = '__all__'


class PrintJobSerializer(serializers.ModelSerializer):
    created_by_username = serializers.CharField(source='created_by.username',
read_only=True)
    template_name = serializers.CharField(source='template.name', read_only=True)
    printer_name = serializers.CharField(source='printer.name', read_only=True)
    asset_count = serializers.IntegerField(read_only=True)

    class Meta:
        model = PrintJob
        fields = '__all__'
        read_only_fields = ['uuid', 'status', 'pdf_file', 'created_at', 'completed_at']


class PrintLogSerializer(serializers.ModelSerializer):
    printed_by_username = serializers.CharField(source='printed_by.username',
read_only=True)
    asset_name = serializers.CharField(source='asset.name', read_only=True)

    class Meta:
        model = PrintLog
        fields = '__all__'
```

---

5. URLs Configuration

5.1 `backend/scanner/api/urls.py`

```
# backend/scanner/api/urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import (
    PrintTemplateViewSet,
```

```
    PrinterViewSet,
    PrintJobViewSet,
    PrintLogViewSet,
    print_label_direct
)

router = DefaultRouter()
router.register(r'templates', PrintTemplateViewSet, basename='print-template')
router.register(r'printers', PrinterViewSet, basename='printer')
router.register(r'print-jobs', PrintJobViewSet, basename='print-job')
router.register(r'print-logs', PrintLogViewSet, basename='print-log')

urlpatterns = [
    path('', include(router.urls)),

    # Endpoint direct pour impression single asset
    path('print-label/<int:asset_id>/', print_label_direct, name='print-label-direct'),

    # Endpoint pour génération QR (existant)
    path('assets/<int:asset_id>/regen-qr/', regenerate_qr, name='regen-qr'),

    # Endpoint scan (existant)
    path('scan/<str:uuid_token>/', resolve_qr, name='scan-resolve'),
]
```

5.2 `backend/config/urls.py`

```
# backend/config/urls.py (extrait)
urlpatterns = [
    # ... autres URLs

    # Scanner API
    path('api/v1/scanner/', include('scanner.api.urls')),

    # Page publique scan
    path('scan/<str:uuid>/', public_scan_result, name='public_scan'),
]
```

---

6. Workflow Complet

```
┌──────────────────────────────────────────────────────────────────────┐
│                WORKFLOW SCAN/PRINT COMPLET                           │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  CRÉATION ASSET                                                      │
│  ──────────────                                                      │
│  POST /api/v1/inventory/assets/                                      │
│         │                                                            │
│         ▼                                                            │
│  Signal post_save → QRCode créé                                      │
```

```
│         │                                                    │
│         ▼                                                    │
│  generate_qr_image() → media/qr_codes/                       │
│                                                              │
│                                                              │
│  IMPRESSION ÉTIQUETTE (Single)                               │
│  ────────────────────────                                    │
│  /admin/assets/<id>/ → Bouton "Imprimer"                     │
│         │                                                    │
│         ▼                                                    │
│  GET /api/v1/scanner/print-label/<id>/                       │
│         │                                                    │
│         ▼                                                    │
│  PDF généré → Téléchargement navigateur                      │
│         │                                                    │
│         ▼                                                    │
│  Impression thermique (USB/BT/Ethernet)                      │
│                                                              │
│                                                              │
│  IMPRESSION BATCH (Multiple)                                 │
│  ──────────────────────                                      │
│  /admin/assets/ → Checkbox → "Imprimer Sélection"            │
│         │                                                    │
│         ▼                                                    │
│  POST /api/v1/scanner/print-jobs/generate/                   │
│  Body: {asset_ids: [...], template_id: 1, copies: 1}        │
│         │                                                    │
│         ▼                                                    │
│  PrintJob créé (status: pending)                             │
│         │                                                    │
│         ▼                                                    │
│  Celery task: generate_print_job_pdf.delay(job_id)           │
│         │                                                    │
│         ▼                                                    │
│  PDF généré → Sauvegardé dans media/print_jobs/              │
│         │                                                    │
│         ▼                                                    │
│  PrintLog créé pour chaque asset (audit)                     │
│         │                                                    │
│         ▼                                                    │
│  GET /api/v1/scanner/print-jobs/<id>/download/               │
│         │                                                    │
│         ▼                                                    │
│  Impression thermique batch                                  │
│                                                              │
│                                                              │
│  SCAN ASSET (Terrain)                                        │
│  ─────────────────                                           │
│  Scanner USB/Bluetooth ou Smartphone                         │
│         │                                                    │
│         ▼                                                    │
│  Code capturé: qr_asset_152_abc123                           │
│         │                                                    │
```

```
│          ▼                                                    │
│   GET /api/v1/scanner/scan/abc123/                            │
│          │                                                    │
│          ▼                                                    │
│   ScanLog enregistré                                          │
│          │                                                    │
│          ▼                                                    │
│   Fiche asset affichée                                        │
│          │                                                    │
│          ▼                                                    │
│   Actions: Ticket / Déplacer / Voir                           │
│                                                               │
│                                                               │
│   AUDIT & REPORTING                                           │
│   ──────────────────                                          │
│   GET /api/v1/scanner/print-logs/                             │
│   GET /api/v1/scanner/print-jobs/                             │
│          │                                                    │
│          ▼                                                    │
│   Export CSV/PDF pour compliance                              │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

---

7. Procédure d'Utilisation

7.1 Configuration Initiale

```
# 1. Créer templates d'étiquettes
python manage.py shell
>>> from scanner.models import PrintTemplate
>>> PrintTemplate.objects.create(
...     name='Standard 50x30',
...     format='50x30',
...     width_mm=50,
...     height_mm=30,
...     is_default=True
... )

# 2. Configurer imprimante
>>> from scanner.models import Printer
>>> Printer.objects.create(
...     name='Samsung SP350',
...     connection_type='usb',
...     is_default=True
... )

# 3. Lancer Celery (pour async)
celery -A config worker --loglevel=info
```

7.2 Impression Single Asset

```
1. Naviguer: /admin/assets/152/
2. Cliquer: "Imprimer Étiquette"
3. PDF téléchargé: label_152.pdf
4. Ouvrir avec lecteur PDF
5. Imprimer → Sélectionner imprimante thermique
6. Paramètres: 203 DPI, Noir seul, Taille réelle
```

7.3 Impression Batch

```
1. Naviguer: /admin/assets/
2. Cocher multiple assets (checkbox)
3. Action: "Imprimer Étiquettes"
4. Choisir template et copies
5. Valider → Job créé
6. Attendre notification (Celery)
7. Télécharger PDF batch
8. Imprimer
```

7.4 Scan Terrain

```
1. Scanner USB HoneyWell connecté
2. Naviguer: /admin/scanner/
3. Scanner QR code sur asset
4. Fiche asset affichée
5. Actions disponibles:
   - Créer ticket maintenance
   - Déplacer asset
   - Voir historique
```

✅ Checklist Finale

Composant	Statut	Fichier
Modèles	✅	`scanner/models.py`
Service PDF	✅	`scanner/services/pdf_generator.py`
Celery Tasks	✅	`scanner/tasks.py`
Management Command	✅	`scanner/management/commands/`
API ViewSets	✅	`scanner/api/views.py`
Serializers	✅	`scanner/api/serializers.py`
URLs	✅	`scanner/api/urls.py`

Composant	Statut	Fichier
Signals QR	✅	`scanner/signals.py`

Ce processus est maintenant production-ready pour une entreprise CMDB Inventory ! 🎉